Chapter 3

Structuring of Distributed Systems

A key challenge is the large-scale structuring of distributed systems. We know that fundamentally all software executes locally and yet must communicate with other nodes in the system in terms of datagrams sent between the nodes. The question is: how do we provide a structured, disciplined framework or infrastructure mechanism that systematizes design of different applications while minimizing the complexity of their implementation yet providing adequate performance, fault-tolerance, security, etc. for the application.

We first consider approaches used in the past (and through to the present) and then consider object-oriented RPC as the current conventional wisdom. Part of the consideration is pointing out some significant issues with the whole remote procedure call approach. We then introduce a new *stateful proxy* approach as another approach to consider.

3.1 History

Some operating system research predating significant distributed systems work focused on the *message-based operating system*. Examples include Per Brinch-Hanson's RC4000, Thoth (one I was involved with), RIG (leading to Mach and eventually to Apple's OS X), Chorus, Amoeba and many others. This work sought to allow operating systems to be structured modularly, with services implemented in different processes rather than in a single "monolithic" kernel. This work is also known as focused on defining a so-called *micro-kernel*, a minimal protected component. There are various commercial offering of this technology, primarily in the real-time embedded control, e.g. Wind River, QNX, Neutrino, etc.

Aside: relative to mainstream operating systems (with the benefit of hindsight), this work suffered from the limited memory and performance of systems versus a better than expected ability for industry to get large operating systems kernels to be reliable. However, with Microsoft blaming most of the Windows operating system crashes on device drivers and the performance issues of these systems for the increasing real-time requirements, such as playing video and audio, it may be we are now (finally) seeing more incentive to move to the micro-kernel direction. Actually, the hypervisors that are the key underpinnings of virtualization are similar to microkernels, but have the key property of providing a virtual machines rather than a "process" environment.

With the advent of high-speed networks such as Ethernet and token ring in the late 1970's and early 1980's, several research efforts in this area moved to implement message-based operating systems in a distributed setting over the LAN. In addition, there were OS folks that just took conventional systems such Unix and modified the kernel to operate over the LAN. Finally, there were so-called *network operating system* projects which attempted to provide a layer of software over conventional systems to make these systems appear to function as one system, at least to some degree.

3.1.1 Distributed Shared Memory (DSM)

Referring back to the previous chapter, one could provide all system services in a distributed shared address space and simply page in code and data, and execute the service locally. This appears to work for *some* relatively small scale parallel applications.

As we saw in the previous chapter, DSM has real limitations in trading off availability versus consistency and providing adequate protection, so at the very minimum, it is not a solution to the issue of a general-purpose framework for building distributed applications and solutions.

DSM also just supports data shipping; there are several reasons for wanting to do function shipping, i.e. call remotely rather than page the data in and execute the operation locally.

- Performance: the relevant data and code already cached remotely which the remote node can use, plus with execution on that node, with contention between multiple clients is local to remote node. This is basically the case where it is better to use *function shipping* rather than *data shipping*, given the costs.

- Remote hardware resources — non-migrateable at the hardware level, including remote processors for parallelism, special peripherals, etc.

- Remote software/data resources — licensing, security, node differences, etc. prohibit migration.

Examples:

- Database server — large cache on dedicated server machine. E.g., faster to do remote keyed file query than transfer index and record file to local node.
- Communication/IO module — married to remote (hardware) resource, e.g., display, keyboard, serial line, etc.
- Authentication server — only authorized or licensed for the remote server machine. Allowing an authentication service to be data shipped locally would clearly open up unacceptable opportunities for subverting the authentication.

3.1.2 Messages: A Communication Abstraction

The common model of messaging is a model of non-blocking asynchronous messages between processes using basic message (send/receive) communication primitives.

- SendMessage(msg, recipient)
- ReceiveMessage(msg)

Early message systems include: RC 4000, DEMOS, Accent, RIG, Mach.

Ideally network-transparent communication provides:

- same message primitives
- same communicating entity identification
- same protocols

whether endpoints are local, remote, and no change if endpoints move. I.e. same programs/processes can be executed locally or remotely, with no change, as done in V.

Advantages of messages:

1. Close to hardware base. I.e. messages similar to network packets, so relatively easy to implement.
2. Flexible — e.g., asynch with heavy-weight processes plus not constrained by the programming language from variants like multicast and datagrams.

3. Synchronization — messages are synchronized on reception by the explicit receive operation of the receiving process, versus RPCs that are invoked as packets arrive.

The anthropomorphic aspect of the message model, similar to people communicating in an organization by memos, is also appealing.

These facilities are largely designed around supporting the direct communication model of state sharing.

Cons:

Messages are an awkward abstraction for programmers who otherwise are purely dealing with procedures and (shared) data structures. This leads to a semantic gap between programming language model of objects (data structures and procedures) and the process/message primitives. In effect, pretty-boy message primitives are what might be called a user-friendly assembly language, at least apparently friendly until you try to use them.

Typical message code:

```
. . .
senderId = receiveMessage( &msg );
switch( msg.type ) {
    case MISSILE_UPDATE:
        /* Cast to missile update record. */
        MissileUpdateMsg * mm = (MissileUpdateMsg *) &msg.
        /* Locate missile record. */
        Missile * m = locate( mm->id );
        m->x = mm->x; /* Update x coord. */
         . . .
 }
```

That is, there is a lot of non-type safe ad hoc marshaling and demarshaling of values into and out of messages.

The empirical experience with OS and application level programming directly with messages has been relatively unsatisfying. Actual use of asynchronous messages tends to put a substantial amount of program state "in flight" and makes the program very difficult to get right. For example, how does a program remember what messages it has sent and what it should do in response to an error on some action it has instigated by one of these earlier messages? A program can also suffer from deadlock when it exhausts system message buffers by sending out too many asynchronous messages. As a consequence, many systems evolve to packaging of services in hand-generated procedure/module interfaces, which forces a strict request-response use

of these message facilities[1]. Then, the cost of supporting non-blocking messaging in buffering, complexity, etc. is significant compared to an optimized request-response mechanism, and the former flexibility is not even used once the applications go to the procedural encapsulation we mentioned above. Finally, maintenance of the hand-generated stub and message formats can become a significant problem unless there is suitable run-time support.

Performance has been a significant problem with message systems in general. The design of message primitives has a significant effect on their cost. There have been lots of problems with inefficient, overly complex message systems in the past. Overbuilt message systems put too much system design into the message system, making it slow or limited and not providing the needed functionality for key application uses, such as RPC. Examples of fancy features include message screening, fancy port/capability-based addressing, synchronization and logging. These "bells and whistles" are not in a normal programming language and thus do not tend to get used from a program, i.e. why would the stub compiler writer put this stuff in when its not in the language, and there is enough challenge to support what is in the language, such as complicated data types.

RPC provides guidance to message primitives, namely RISC-like thinking: Do not put fancy stuff in the message primitives that cannot or will not be used from the language level, or is not cost-effective for the language-level uses.

3.1.3 UDP/TCP: Socket Abstraction

Another approach is to build on the socket abstraction of Unix (and now Linux), relying on the local host communication for local interprocess communication. To some degree, this is just another communication abstraction similar to messages.

Using this approach with UDP/TCP, the application/system can then operate over the Internet, at least in theory. Moreover, you can build on the familiarity of the socket abstraction given they are used in other communication applications. Also, compared to the earlier times, increased processor speed and improved socket implementations make this approach reasonably efficient.

As disadvantages, this approach is similar to the problems with messages in requiring applications deal with marshaling and demarshaling, mapping onto endpoint addresses. However, in many respects this abstraction is far worse than messages.

[1] Another view of this arises as follows: If a process needs some data, it requests it in a read and necessarily waits until its gets the response, so this is request-response. If a process is writing/sending some data to another process, it typically requires confirmation that the data was received and accepted before deleting its local copy so, again, the communication becomes request-response

TCP provides a byte stream abstraction that throws away the record or PDU boundaries, which are important semantics for some applications. You then need to introduce your own framing protocol on top and struggle further to synchronize with the actual packet level, if those semantics or the performance is important to you. (I.e. the packet level provides the behavior that either the whole unit arrives or none of it does, i.e. atomic data delivery.) Fundamentally, TCP was designed 30 years ago to support telnet, not distributed systems.

On the other hand, UDP provides a datagram or record-like communication abstraction but does not provide reliable delivery, so requires a separate mechanism to provide basic reliable transport unless the application does not require this, an unusual case. Thus, you are stuck between trying to recover packet semantics onto of TCP or building your own transport protocol on top of UDP.

I think a far better communication abstraction would be a packet-based transport protocol along the lines of XNS Sequence Packet Protocol, but extended with real-time and multicast support along the lines described in Liang 2002/3 (who also looked into framing extensions to TCP). That is, a transport protocol can have the capability of optionally skipping over missed packets rather than retransmitting, particularly for real-time communication. Similarly, a TCP can be extended to support multicast with a relative small increase in size of code.

3.1.4 Distributed File Systems: The File Abstraction

Interprocess communication can be implemented through files (and pipes). For example, one process (such as a first pass of a compiler) can write its results to a file, and a second process (i.e. the next pass of the compiler) can read this file to handle this as its input. In this vein, the original Unix system was a surprisingly elegant approach, providing a generic file I/O access model in terms of open, read, write close and seek. Ken Thompson describes the Unix kernel at one point as an I/O multiplexor, implementing this abstraction across a variety of files and devices.

Building on this abstraction (and code base), Locus, Sprite, MOS and several others were distributed Unix systems that provide a distributed file system, and thus with interprocess communication using files. Subsequently, Lucent Technologies (as the same basic organization that developed the original Unix) came out with Inferno, a distributed operating system that provides a distributed file system abstraction and is designed for use with a non-object-oriented interpreted language called Limbo, as a competitor to Java (but without objects). (This system and language seems to have largely lost out to Java.) It appears to try to carry the file I/O abstraction of Unix to a networked and interpreted world. Unfortunately, I think they really missed the object-oriented bandwagon. Files are one type of object and not anywhere near as powerful as providing a general object-oriented facility, as with CORBA, Java RMI, DCOM, etc.

These systems were, in all cases I know, implemented as specialized protocol highly targeted to implementing the specific requirements for a particular version of Unix kernel. For example, Sprite includes a complex protocol to handle the case of a shared file pointer, causing the file pointer to be moved back to the original host of the file. These protocols are not documented in the vein of Internet protocols, but treated as internal implementation details to the distributed Unix kernel implementation. I.e these systems are basically a distributed Unix Kernel in the sense of a Unix kernel interface extended across network to multiple machines. Any attempt to document the protocols would reveal tremendous coupling to internal Unix kernel implementation details. (My effort with a protocol called VMTP struggled with this problem.)

A (distributed) file system, a high-level of communication model, is an adequate form of communication and state sharing for a large class of applications. Eg. document production, software development, data processing. However, there are several limitations:

- File object abstraction appears clumsy and error-prone for general distributed application programming. I.e. how do I program a business service on a remote server? You write ad hoc messages to a file and I read the file — gag! This is like messages, but through files, with all the associated marshaling and demarshaling issues. (However, this is basically how much of the web operates.)
- Performance: If the application does not distribute using the file interface, communication is often more expensive because of the indirection and overhead of file access to share data. For example, consider using a file or a directory as a queue as is done with the Unix printer facility.
- Consistency: Files generally provide coarse-grain or time-based consistency, making consistency difficult to handle. For example, some distributed file systems just check on open to see if the file has been updated whereas others, such as NFS, check on a page basis on access after some timeout period. Any application requiring finer-grain interaction or consistency is difficult, clumsy or inefficient to write on this model. In particular, if the unit of consistency is a file, any update to a file calls for the whole file being retransmitted.

With the benefit of hindsight, I would say these systems were not broadly compelling because they were necessarily too tied to particular Unix semantics to scale or to be open and standardizable, and if they were not, they would have lost one of their key advantages, namely the ability to run Unix applications unmodified. That said, I understand some distributed Unix systems have been achieving some success on Wall Street because various firms have large financial applications that were written as multiple processes running on a single "system", and the memory and processing

requirements have outgrown fitting onto a single blade. These distributed Unix systems preserve this single system abstraction while allowing each process to run on a separate node. Thus, there is some success but this approach does not seem likely to be significant in the mainstream. One example is Mosix. Another was Qlusters, although the latter company appears to have scaled back to just focus on an open-source cluster resource manager.

In general, the large amount of error-prone network/communication layer programming with these approaches and the limitations of the distributed file system approach for general applications has lead to focus on providing *remote procedure call*(RPC). Mixing that with the trend to object-oriented programming, you see OO RPC in recent systems such as Java RMI and C# .Net framework. This direction is discussed next.

3.2 Object-oriented RPC Approach

A strong current direction with distributed systems is developing distributed applications on top of a middleware layer such as CORBA, Java RMI/Enterprise Beans, Web Services (XML), Microsoft's DCOM, or more recently the so-called ".Net" framework. All of these are object-oriented (to some degree) frameworks that provide a base set of services and allow applications/services to be developed as application-specific or service-specific objects.

With these middleware systems, you specify object interfaces, often in terms of an *Interface Definition Language*(*IDL*). The middleware then generates the code that deals with distributed operation so you just need to implement these interfaces, the same as local OOP programming, more or less.

Object-oriented structure is used because it is popular, powerful and probably necessary to handle distribution cleanly, given that you need specified interfaces, you need to support multiple implementations and multiple instantiations of the same interface and you need a uniform means of referring to objects across the distributed system.

There are three key aspects to object-oriented technology:

1. A *uniform reference*, naming or identification scheme is used for all objects, allowing objects to be named independent of type.

2. Client software of an object accesses the object through one or more *interface*s, that define what is visible to the client, with *dynamic dispatch* to the implementation encapsulated behind the interface in each case, allowing for multiple different implementations. For example, there may be multiple different file server implementations.

3. The *inheritance* mechanism, allowing an interface type to be an extended version of another interface type (and also allowing an implementation to provide an implementation of an interface.)

These aspects are combined in the object-oriented procedure call, in which the call is dispatched to the object implementing the interface, and the actual procedure to execute is determined in the context of that object.

This object-oriented technology can be used effectively to structure the state (shared and non-shared) of a distributed system. For example, a *shared memory* is a (distributed) object supporting two primary member functions, as expressed below in CORBA IDL:

```
interface Memory {
public:
    Datum read( Address ) const;
    void write( Address, Datum );
    . . .
  };
```

that behave like a conventional memory module in terms of read and write behavior. (These operations are also called `fetch` and `store`.)

In a distributed system, the interface aspect of object-oriented technology can be applied to shared state to provide application-specific interfaces, along the lines introduced in the last chapter. The encapsulation of implementation behind these interfaces allows the use of application-specific consistency mechanisms, transparent to the client software.

An object in a distributed system requires some form of designation beyond that of the simple virtual address normally used in single-address space programming. In an object-oriented design, this designation is referred to as the *object reference*, the uniform object reference referred to earlier. I advocate objects being named in a global (system-wide, at least) character-string name space. In addition, the system may support efficient internal naming of an object with which the client is in active communication. In this mode, it identifies the object with a transient fixed-length 32-bit identifier in the context of a communication channel to the server. The issues of naming and identification are discussed further in Chapter 5.

The object-oriented procedure call is extended to work over the network using *object-oriented remote procedure call(*OO RPC) so a thread on one machine to can invoke a member function or "method" on an object on another machine, using the same basic syntax and semantics as a local call. For example, with OO RPC in C++

```
File *  f = . . .
int bytes = f->read( buffer, count );
```

the file referenced by `f` can be remote on a file server. Transparent to the client code, the read call is transmitted over the network to the file server, returning the data back over the network to the caller.

A pointer to the remote file object could be obtained by code similar to this.

```
String name = . . .;
// Lookup object in directory by ‘‘name’’
Ptr<NamedInterface> o = Directory::root()->interface( name );

// Cast pointer to a file from a raw pointer if actually a file pointer.
Ptr<OpenFile> f = dynamic_cast<OpenFile *> o.rawPointer();
if( f == 0 ) { // Not a file
   return an error
}
```

Ideally, OO RPC is provided as an extension of an object-oriented programming language such as C++, Java or C#. However, a somewhat awkward object-oriented interface can be provided in a non-object-oriented language such as C. We generally assume the use of C++ in this discussion.

A key property of object-oriented (remote) procedure call in distributed systems is dispatching on the object first, rather than binding to a particular procedure. Dispatching on the object allows there to be, simultaneously, multiple different implementations. For example, the open file object referenced above may exist on one file server whereas a second open file may be located on a separate independent file server. In contrast, in a system like Unix, the file open, read, write and close routines dispatch directly to the single local operating system kernel implementation, precluding direct dispatch to independent implementations. (Unix allows dispatch internal to the kernel to different file systems, using the so-called virtual file system (VFS) interface.)

3.2.1 OO RPC Implementation

OO RPC is implemented as a local call to a *proxy* object (Shapiro86), which stands in for the "real" object. The proxy object translates the call into a call or request message to the node containing the real object and forwards the call to the remote real object to which it is bound, as depicted in Fig. 3.1. Thus, the remote execution of the call is network-transparent to the caller. The proxy object has also been called a *surrogate*.

Conventionally, the remote procedure call itself is implemented by an IDL or *stub compiler* that takes an interface definition as input and generates a set of proxy routines for the client side and stub routines for the server side. A proxy routine, when

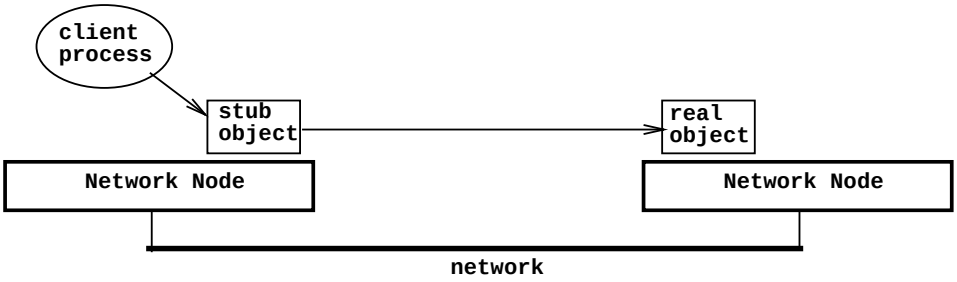

Figure 3.1: Object-oriented Remote Procedure Call

called, formats its parameters into a message (called *marshaling the parameters*), maps the object identifier to a network address, sends the message over the network to that address, waits for the response message, translates the response message into return values and returns these values to the caller. The stub compiler also generates, for each interface, receiver routines that receive a call message, *demarshal* it into a procedure call on the stack, invoke the designated call, marshal the return value into a response message and send the response message back to the caller.

Similarity to the System Call The remote procedure call can be viewed as an extension of the system call mechanism of conventional operating system. The basic steps of a system call are as follows:

- *Stub routine* assembles parameters and executes machine trap operation, causing the processor to switch to privileged mode and branch to a trap-handling routine.
- Trap-handling routine moves parameters to 'system' stack and calls specified system routine with these parameters. (This usually changes the memory addressable by process as well.)
- On return, return values are passed back and system privilege is relinquished.

The cost of system call is much more than that of a normal procedure call, often 100 times or more the cost of a simple procedure call.

With system calls, there is one distinguished set of protected, privileged procedures. Remote procedure call allows each new service to define new calls, as described next. (Relatedly, an issue with distributed systems is an explosion of different interfaces, compared to the highly restricted number of "system calls", primarily because only a few "privileged" people are allowed to add system calls to an operating system.)

Dynamic Invocation In a single-address space program, all types used in the program are normally defined in the program, and thus known to the run-time type identification system. However, in a distributed system, a program may in theory receive a reference to an object of a type not defined in the program. To address this problem, an interface and type repository is provided that stores the typing information on an object. For example, in CORBA, the interface repository maintains a hierarchical structure identifying modules, interfaces within modules, constants, typedefs, exceptions, attributes and operations within an interface (an attribute is a logical variable of the interface that can be read and written).

The next step beyond determining the type of an object reference received at run-time is to be able to invoke operations on that object. The conventional view is

that one just downcasts to the expected more-specific type and calls the appropriate functions. This approach requires that the proxy implement the more specific type. It may require that the code for the type-specific proxy be loaded at the time the reference to the proxy is received, which is feasible using a dynamic linking facility.

An alternative approach, called the *dynamic invocation* in CORBA, requires in general the ability to dynamically (i.e. at run-time) create a call to the object, so does not require that the type-specific interface was compiled into the calling program. This facility is implemented by what might be regarded as an interpreter version of the stub compiler.

This facility, however, makes limited sense to me. If you do not know the interface or semantics of a service at the time you write the client, I do not see how you can generate a call to the service and know what it does or how to use the result. If the interface is known, it can be compiled in and you do not need the dynamic invocation capability, as defined in CORBA. The primary legitimate use of this facility as I understand it is interfacing to CORBA objects from scripting languages and user interfaces which are typeless environments, so type-specific calls need this support.

There is an intermediate approach in which the middleware dynamically discovers that some referenced object is a specialized implementation type and dynamically loads that implementation for the corresponding proxy. In this case, the application is hard-coded with the interface for this proxy, typically down-casting from a more generic type, but is oblivious to the specific implementation type. That is, the implementation type is dynamically discovered but does not affect the application logic in any way.

Additional aspects of implementing RPC are considered in Chapter 13.

3.2.2 Using (OO) RPC to Define a Network Service

With a remote procedure call facility, the steps to providing a network service are fairly simple:

1. Design the service *interface* or interfaces and then generate the client proxy and server receiver routines, or select an existing one, e.g. a new file server implementation using an existing interface.

2. Implement server receiver routines in server module and a means to cause it to be executed and register itself in the system name space.

3. Provide the client stubs to applications, allowing them to access this new service.

Example: file system on machine A *exports* a set of routines that are the file system operations. Clients on other machines invoke these as remote procedure calls.

The above steps assume that there is already a directory service that this new service can register in, allowing it to be located by clients.

OO RPC effectively partitions the implementation of distributed systems into three pieces, namely:

1. OO RPC implementation — the basic call mechanism, namely the transport management, binding and marshal/demarshal

2. System Objects and Services — standard library/OS facilities

3. Applications - the part you get to write

In particular, the applications and system services are cleanly partitioned from the packet level details.

3.2.3 Example RPC Systems

Here are a few example distributed object-oriented systems.

Microsoft COM/DCOM/COM+ The Microsoft DCOM, extended from COM and more recently in COM+, provides a distributed object framework as an extension of the OLE (Object Linking and Embedding) facility. OLE allows objects to be linked by reference between types of documents and also objects to be embedded in other objects. For example, a spreadsheet graph can be embedded as a figure in a text document. An attempt to edit the figure inside of the document text automatically and more or less transparently invokes the spreadsheet program to operate on the spreadsheet graph. That is, the edit operation is interpreted based on the type of object being edited, just as with a procedure in a conventional object-oriented environment.

DCOM extends the COM model over the network with an object-oriented RPC facility, an object model based on objects derived from a common interface type with network-wide naming, plus distributed security and transaction facilities. DCOM also incorporates a threading facility, supporting the implementation of multi-threaded servers (but this facility is also for use locally).

COM+ is DCOM and COM integrated into a C++ programming environment.

CORBA The Common Object Request Broker Architecture (CORBA) is a source interface standard promoted by the Object Management Group (OMG), an industry standards consortium. CORBA supports dynamic invocation from object interface provided by the Interface Repository as well as statically compiled stubs. An *object adapter* module provides management of object references (including registering implementations and mapping to object implementations), object and implementation management (including activation), and *method* invocation on objects. Various forms of object adapter are anticipated. For example, there can be an object-oriented data base adapter that provides access to data base objects as well as a more conventional basic object adapter that supports calls to objects defined and implemented by a set of applications. The CORBA Interface Definition Language (IDL) is a subset of ANSI C++ with some minor extensions for specifying the access modes of parameters (i.e. in, out, and inout) and for datagram calls. Basically, the CORBA IDL looks approximately like a class declaration with the `class` keyword replaced by `interface` and no inline functions in the class declaration. (The IDL also currently prohibits overloading of member functions.)

More recently, Microsoft has been promoting the .Net platform which folds in "web services", which is largely redoing RPC using XML.

Java RMI Java *remote method invocation*(*RMI*) is the RPC support in Java, which builds on object serialization to marshal and demarshal calls. Java supports passing objects in general, not just references, and advertises being able to "pass behavior". This is implemented by passing a serialized representation of the object implementation, relying on Java's JVM implementation and run-anywhere approach. For security, it also relies on Java's protection in the form of garbage collection, similar to its use with applets. It's not clear how compelling this feature is over just downloading an applet when local program/applet execution is desired.

Java RMI requires that interfaces that are remotely accessible be derived from the `Remote` interface.

HAVi The Home Audio Video initiative (HAVi) was an industry effort to create an object-oriented standard for connection and interoperation of home entertainment devices, such as CD players, DVD, TV receivers and displays, amplifiers, etc. The focus was on connection of these devices by IEEE 1394 networking, a 100 Mbps technology that supports both asynchronous and isonchronous traffic.

HAVi defines identifiers, APIs and a messaging system that includes event notification. It allows a device to be controlled by a limited set of standard operations or by uploading a device-specific Java code unit, called a *havlet*, that implements a proprietary protocol to control the device. Everything is nicely modeled as objects.

HAVi addresses a real problem. Currently, home entertainment systems operate in the 1950s with proprietary point-to-point ad hoc interconnects for everything. A typical home theater system is lots of wires yet provides limited control, flexibility and customizability.

However, as a criticism, this effort may be trying to preserve a basically dead world — trying to put wheels on a horse with the hope it can compete with a car. All the input and output devices of a home entertainment are basically just computer peripherals. Ethernet is a well-established physical networking technology, and anyone that can afford HAVi in the short-term will have Ethernet in their home for (shared) Internet access. So, all the various home entertainment devices plus security devices and other home system devices (air conditioning, lighting) will be connected to one or more computers that will be connected by Ethernet, running Internet protocols. The attempt to create value-added, high-margin peripherals is understandable from a business standpoint, but seems unlikely to succeed, given the momentum of the Internet approach, and the falling cost of PC hardware and Ethernet. (Also, the need for fancy isonchronous support is a myth for 100 Mbps Ethernet and beyond - gigabit Ethernet in the home will come!)

As a further concern, the HAVi approach allows every device to provide its own interface, both for user and program, which seems excessively flexible and leads to unreasonable system complexity.

DOE Distributed Object Everywhere (DOE) was a distributed object framework developed by Sun Microsystems. DOE executes on top of the Distributed Object Management Facility (DOMF), which is SUN's implementation of a CORBA-compliant object request broker (ORB). The primary emphasis is on unifying the handling of documents and similar objects across multiple applications in a typical office environment. For example, rather than have a spreadsheet program tightly coupled to a particular GUI interface, and only accessible through this interface, the spreadsheet program can be viewed as the methods or operations on files (i.e. objects) of type spreadsheet. When an operation is invoked on a spreadsheet object using a program API, the (back-end) spreadsheet server is invoked to perform the operation on this object.

I believe DOE suffered from lack of application buy-in, in comparison to MSFT, which can get a strong application presence simply by putting the "Office" products on top of it. Moreover, it is difficult for independent application developers to take advantage of a higher-level API while remaining platform-independent unless something is available on other Unix platforms.

WWW Examples You might regard the WWW as objects from some standpoint. The Web objects are identified by URLs, primarily using HTTP to access and HTML

for (re)presentation, extended more recently with GIF, TIFF, JPEG, VRML, Java, etc.

The whole direction with Java and JavaScript introduces some stronger sense of objects into the Web world. For example, JavaScript allows you to write Java-like programs that manipulate objects that are created inside the browser as a result of downloading the associated page. Each image, frame and such like is an object that JavaScript allows you to manipulate. However, it is hardly distributed in the sense we are discussing.

In many ways, the Web is a step backwards in technology from a protocols and distributed systems standpoint.

V Kernel The V system was designed along object-oriented lines but without the benefit of an object-oriented language, hand-coding stubs on top of messages. The UIO (uniform I/O) interface was one key aspect of the object-oriented structure.

One can view the distributed operating systems kernel as providing a base of software facilities for recursive structuring that provides the special case, terminating the recursive structure, as suggested in Fig. 3.2. Kernel should be small, fast, secure, reliable with functionality required to build rest of system. It should be the base set of objects on which everything else is built. So, how to define kernel functionality?

Backplane View One approach is to view a distributed kernel as a software chassis, drawing the analogy with the way a modular hardware system is built. That is,

- a standard backplane or chassis provides slots, communication and power.
- individual boards implement standard functions with board size, communication and power compatible with backplane.
- system configuration is achieved by plugging in desired boards for desired services and capabilities.

Using this analogy, the corresponding software objects are as follows:

- an address space corresponds to a slot in the chassis.
- IPC and RPC facilities correspond to chassis or backplane communication.
- processes and processor allocation correspond to backplane power.

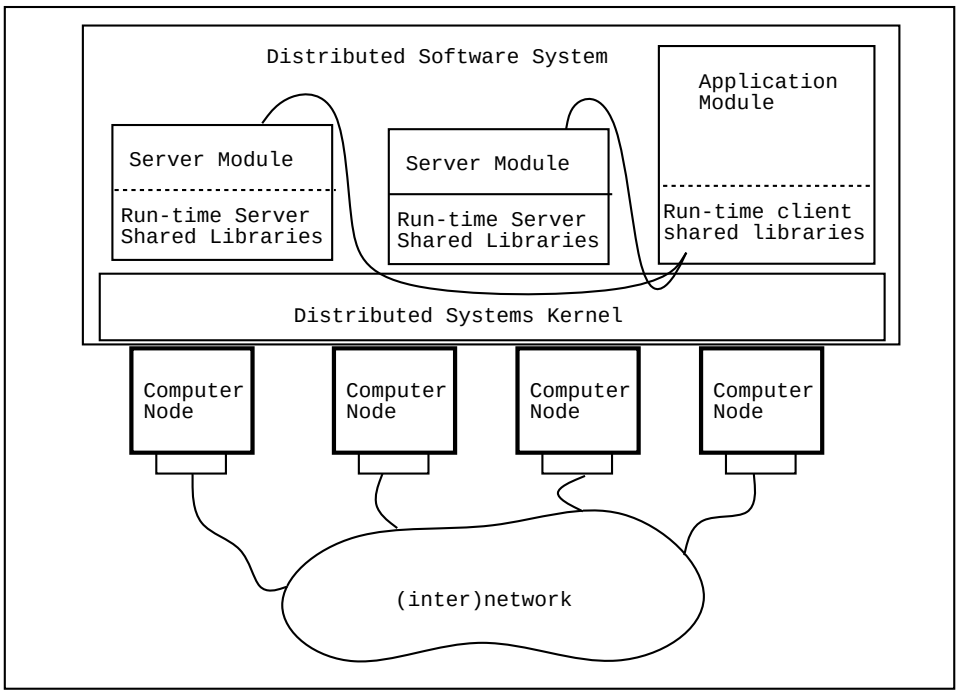

Figure 3.2: Kernel as Base for Recursive Structure

Following the hardware analogy, the kernel only provides these base services, and no others. The other services are provided by modules that plug into the slots, i.e. programs running in unprivileged address spaces.

The V distributed system is one example of a distributed operating system designed along these lines, as part of research to understand kernel interfaces and higher-level service interfaces, etc. In V, the kernel provides a message-based interprocess communication facility plus process, memory and device management. Low-level process, memory and device management operations are handled by (pseudo) servers or manager modules implemented by the kernel as the base case of each of these facilities. Higher-level process, memory and device management is implemented by separate server programs that invoke operations on the base manager modules as part of their implementation, i.e. recursively. Moreover, these kernel base services can invoke services on other higher-level services, completing the recursive structure. For example, a page fault can result in a kernel memory manager calling a file object at a higher level, typically on another node. The kernel is also simplified by exceptions being forwarded out of kernel using IPC to user-level handling modules. A significant problem with recursive structuring in the V design was that the RPC-like behavior was implemented on a blocking message facility and process-identified server process structure. As a consequence, recursive calls tended to deadlock. A pure OO RPC approach addresses some of these deficiencies.

3.2.4 Advantages

The OO RPC-based approach has several strong advantages for building distributed systems. Basically, it provides a network-transparent environment for application logic using constructs, namely objects and procedures, that are familiar to application programmers. In more detail, using OO RPC, a common set of wire protocols, including the presentation, session and transport protocols are provided by the OO RPC system, avoiding the ad hoc message structuring that is common with older approaches. Second, the complex protocol code is normally automatically generated from interfaces by the stub compiler, allowing programmers to focus on the service or functionality and ensure that the same session, transport and presentation protocols are used[2]. Finally, it provides a high-degree of network transparency, especially when provided as an extension to an object-oriented implementation language such as C++, Java or C#. It is transparent at the language source level whether an object is local or remote. The same source code can deal with both situations.

[2]The alternative of PDU or message-level programming generally leads to a lot of ad hoc handling of these layers, similar to the diversity of calling conventions that are used in assembly language programming. Consider the Internet protocols such as ICMP, IGMP, BGP, etc. for example.

RPC versus Messages

Let's compare RPC to a message facility again. (Note: procedure calls on objects are called *messages* in some object-oriented languages, but that is not what we mean here.)

RPC implies a thread logically executes a call in the remote address space, rather than explicitly handing a message to a thread. However, in the implementation, an RPC is executed by an "anonymous" thread acting on behalf of the caller, that is activated with a message containing the call record. That is, the process/message model is "assembly language" of the implementation, not part of the interface. The execution of the remote call is then synchronized with the execution of other threads in this remote address space using standard shared memory primitives, such as compare-and-swap, locks, semaphores, monitors, etc.

Many message systems evolved into providing (hand-generated stubs) to applications for the systems services, e.g. V. Thus, the key innovation of RPC is providing automatically generated *stub* routines that translate procedure calls to message primitives.

However, the procedure model from programming languages tends to impose limits on the message functionality. I.e. no multicast, no datagrams, no asynchronous streaming support. Extensive network-level programming with PDUs will not go away until there are widely available OO RPC systems that support these extensions or we come up with a better framework/infrastructure alternative.

The above provides the base advantages of RPC. While there is increasing use of OO RPC with CORBA, DCOM and Java RMI, there are a number of outstanding problems that raise serious questions about RPC in general.

3.3 Problems with RPC

RPC actually has a number of problems that are severe, i.e. in my opinion, insolvable to the degree of they make it the wrong approach for many distributed systems.

3.3.1 Performance

The performance problem is not strictly that RPC is "slow" but that network transparency introduces a quasi-hidden cost that can vary by 5 orders of magnitude or more. For instance, a procedure called locally may take less than 1 microsecond to complete whereas called remotely, it may take 10 milliseconds in the normal (remote) case. With network congestion or packet drop, the cost can go (much) higher, and of course with network failure, the cost can become bounded only by some long

timeout parameter, which can be seconds or longer. This cost can be multiplied further by considering an iteration through a collection if each call in the iteration is remote.

It seems completely infeasible to engineer a system for predictable performance when the cost of basic procedure call can vary by this extreme amount unless both the network delay can be bounded and the procedure calls that are remote can be identified and programmed "specially" — so much for network transparency.

However, as systems scale and encounter difficult environments, it is impossible to bound network delays. Moreover, roundtrip time is the one thing that is not going be to reduced by improvements in technology, so sayeth Albert Einstein.

This round-trip time is further magnified if you are forced to use the unicast request-response model of RPC in cases where datagrams, streaming and multicast are more appropriate — all conventional optimizations used with Internet protocols. As an extreme example, consider trying to use RPC to send real-time video to a number of clients over a network. Rather than using multicast and datagrams as would be the obvious choice starting with a sockets abstraction, you would be forced into blocking unicast request-response model with RPC incurring all the attendant overheads.

In servers, there is often a large number of clients being served simultaneously so you are less concerned with delay and more with the throughput of serving clients. Thus, a server can use a large number of threads to deal with the delay of RPC. For example, a web server that needs to request state from a session server using RPC can have a thread per client and expect that the large number of threads can effectively pipeline requests to the session server and achieve the desired throughput. However, threads can be expensive in space (and operating systems overhead in some cases) and incur extra cost associated with context switching to block and unblock as part of RPCs. I.e. each "event" can entail a thread performing a small amount of computation, generating a request/call message, then blocking, and then on each incoming return packet, redispatching this thread. The processing (and memory access) cost of this RPC blocking and dispatching can dominate the cost of the application processing.

3.3.2 Semantics

OO RPC tries to provide *network transparency*, i.e. applications do not need to know whether a call is being performed locally or remotely. However, local call semantics are: the call is performed or else you receive an error return, or else you die during the call. That is, the node you are running on can crash and you do not know whether the call completed or not but then you do not care because you died. However, with a remote call, it is possible for the remote machine or your communication with this remote machine to fail and you do not know whether your call completed or not, only that you timed out waiting for a response.

The conventional local semantics are normally referred to as exactly once (unless it throws an exception). Distributed systems literature considers *at least once* and *at most once semantics* as well. These are "nice" terms but what do they actually mean?

If a client node tries to re-execute the call after the above timeout, it is possible that the link and/or server has recovered, causing the call to be re-executed. This provides "at least once" semantics but it does mean that a call might be executed more than once. With arbitrary semantics to the call, it is not possible to determine the effect on application semantics but it is obviously not the same as local semantics. For instance, if the action is to debit a bank account, debiting the account twice is not the correct action.

On the other hand, if the client node does not retry the call, it is possible that the call did not take place, providing "at most once" semantics. However, again, with arbitrary call semantics, the application impact is indeterminate. Continuing the above example, if the call is to debit an account, the failure to debit is just as bad (or worse for the bank) than debiting the account twice.

However, the terms "at most once" and "at least once" are nice but mask the danger of trying to build systems with these semantics. Considering "at most once", this term really means "might not happen" in some cases. How can you possibly make sense of software where a procedure call may fail to execute and indicate nothing or a procedure call can succeed and yet throw an exception? These are the two choices given that when a call has been issued and no return or response is received, the implementation either fails silently, hangs or throws an exception.

Similarly, "at least once" is a sanitized way of saying that some calls may execute multiple times without any indication to that effect, or possibly quite a bogus indication. For instance, considering a call creating a new directory. A second invocation of the same call can return a bogus error message indicating that a directory by the specified name already exists. How does one write software to make sense of bogus error messages of this nature or of multiple calls transparently occurring, such as double deposits to a bank account? Moreover, how can you possibly test such a system, given these events occur at random and presumably rare times? I have no clue on how to determine or test that all remote calls have these semantics, especially given that what is remote or not cannot be determined in general at compile time, and may change with configuration and dynamically during execution. The fact that these situations are expected to occur very infrequently just makes this worse[3].

On the other hand, trying to provide exactly once semantics requires maintaining persistent state about the calls between the client and server almost indefinitely to

[3] I refer to this sort of problem in software as a *time bomb* because it may lie dormant for some time and then eventually it will go off.

filter out duplicate calls. (The persistence is required because the server might have crashed right after completing the call. "Indefinitely, because there is no bound on how long a client might retry or a link might be down.) However, this is a significant burden on a communication layer that is otherwise only maintaining transient real-time state.

Another approach is to encapsulate all state modifying RPCs in transactions, leveraging a transaction log to provide this persistent state. Then, a client node, on resumption of communication with the server can check whether a given transaction completed or not and determine whether to redo or not on this basis. Generally, transaction mechanisms have been costly to use for general-purpose RPC, compounding the performance issue.

3.3.3 Language/Procedural Restrictions and Complications

RPC also suffers from its tight coupling to the "procedure call" construct in the programming language because it can be both forced to be too complex in order to support all the possible semantics of the associated programming language and because it is difficult to support various forms of communication that do not fit into the request-response behavior of the conventional procedure call.

On the former point, RPC systems that attempt to provide complete transparency within a programming language end up dealing with all the possible parameters and return values that can be used with the programming language. For instance, the stub compiler trying to support full transparency may have to deal with a procedure that is passed a pointer to a AVL tree by (non-const) reference, forcing it to deal with marshaling this AVL tree to the remote side, allowing it to be modified and then marshaling the modified tree back to the caller, revising the original tree accordingly. Projects that have tried to achieve a high degree of language fidelity of this nature have suffered (to my knowledge) from a very high overhead in development, complexity and bugs with the stub compiler, yet end up supporting RPC behavior that is not desirable in real systems. I.e. you generally want to avoid passing complex data structures around because of the increased exposure to corruption and errors. For example, B. Lampson expressed concern that using OO RPC makes you dependent on the stub compiler writer if you need any changes or extensions to the protocol mechanisms you need (which carries the implication that one size does not fit all with RPC).

Also, such endeavors end up being excessively coupled to a specific programming language, given that different programming languages have slightly different procedure call semantics and exceptions. The solution with systems like CORBA is to introduce yet another language, IDL, in which you specify these interfaces.

True, you can give up on providing this degree of transparency but then you are

giving up on a key claimed advantage of RPC, namely providing a transparent procedure call access to remote objects the same as local objects.

In this vein, OO RPC is not used in the definition of Internet protocols. At present, most protocols are in fact directly defined in terms of wire packet formats. You should keep in mind that the current practice is basically an "assembly language" level approach to protocols at the PDU (Packet Data Unit) level. Part of the reason is legacy people carrying on legacy approaches. However, another reason is that the Internet rightly focuses on having protocol specifications that allow interoperability between independently developed implementations, whereas OO RPC systems have focused on providing a source code and programmer interface, hiding the protocols they use. For instance, CORBA did not even define the protocols it uses originally, just client software interfaces. To allow use of OO RPC as well as Internet-like RFC specification of the protocols required for interoperability, there needs to be a fully specified RPC and presentation protocol that is supported by a widely available OO RPC mechanism, which is sufficiently functional and efficient to use, plus is compatible with a variety of programming languages used in the Internet. One might argue that it also must allow "manual" implementation of the invocation and reception routines that would normally be generated by the stub compiler. Otherwise, any attempt to define an Internet protocol in terms of OO RPC will be frustrated by the inability of some to implement it, because they lack the stub compiler.

On the other side of this coupling to the procedure construct, the procedure call model does not directly accommodate a number of communication paradigms that are proven valuable, if not essential, in real distributed application and systems. Examples include datagrams, multicast and streaming.

A number of research projects have attempted to address these deficiencies. For example, V provided a multicast request-response paradigm by sending to a "group" of processes, allowing multiple responses to be received. Also, the MIT Mercury project looked at supporting asynchronous procedure call by introducing the notion of a `promise`, an object that represents the deferred return value of an asynchronous call. The client initiates the call and then checks the promise periodically to see if the call has completed and collects the result if so. There has been other work looking at passing a "pipe" or stream as a call parameter and using a combination of RPC and streams.

However, all these extensions are just more departures and complications relative to the "procedure call" model, detracting from a key claimed advantage of providing a familiar programming model.

Note: Some people express concern about the difference in semantics between remote calls and local calls because a remote call can fail because the server machine is down. However, with the use of exceptions in OOP languages such as C++ and

Java, applications have to handle exception returns in any case. The problem arises as above in whether the call completed or not.

Overall, using pure OO RPC to implement distributed applications and systems with a strict unicast request-response form of communication does meet not all or even many application requirements. In particular, current Internet protocols use datagrams, multicast and streaming. For example, BGP uses a streaming of (type,length,values) triples that are effectively asynchronous write operations, accommodating the long latencies of the wide-area environment. Conventional OO RPC does not support this streaming.

OO RPC also seems too expensive for applications that currently rely on raw data streaming. Some work in this area, extending and optimizing OO RPC for these uses is discussed by Zelesko and Cheriton (96).

Given all these problems with RPC, you are presumably asking if there is an alternative?

3.4 Alternatives

The common alternatives of messages, distributed files and distributed shared memory have been covered earlier. In summary, I was claiming that, in brief, messages (or any similar communication-level mechanism) is low-level (leaving you to do all the state-related work per application), DSM is a state-oriented approach that is too inefficient, tightly-coupled and lacking in application semantics, while the distributed file API to is restrictive.

It may be that you cannot do better. It also may be that one size does not fit all and you simply have to use the lesser of the various evils for each application. That said, let's consider some other candidates.

3.4.1 Smart Proxies

The notion of an "smart" proxy is to have the proxy object perform extra processing rather than simply map each call in its interface to a call over the wire. In particular, it can implement or partially implement some operations locally, with the attendant performance benefits. For example, a proxy for a remote file may cache the file properties and data so that it can respond locally to file requests rather than incurring the overhead of going over the network to the file server to interact with the real object on each call. In the general case, the specialized class may contain components stub objects for several remote objects and cache data, plus deal with the attendant consistency issues.

As illustrated with the caching example above, the distributed object programming allows the flexibility to make a sensible trade-off between *data shipping*, where the data is moved to the computation, and *function shipping* where the function is sent to the data. In particular, the simple proxy object purely moves the function to the data, i.e. the real object, whereas a programmed object that does local processing can bring the data to the local node (i.e. caching the data), thus shipping the data to the computation. In this model, data shipping is generally an optimization over function shipping[4]. In some cases, this optimization is truly compelling, and needs to be supported to have a practical design. For example, the application-visible file objects need to cache file data to avoid having every file I/O entail network traffic and the delay of a network roundtrip time. In other cases, such as an authentication service, security concerns may preclude moving data to the local node, thus requiring a pure function shipping model. In addition to function and data shipping, there is the flow of data values between objects at the client level. For example, a file data may be transferred from the file to a printer for output. We regard this dataflow as an aspect of the value-oriented programming that typically takes place on top of an object-oriented framework, and a third option to the data/function shipping option. For example, the file data when moved from the file to the printer does not retain any consistency semantics relative to subsequent changes to the file, unlike the data shipping between the components of a distributed file implementation.

Proxy object programming provides separation between the access to the object's client interface and what actually goes over the wire, conventionally referred to as the protocol. For example, the interaction between local file caching object and the real file object would conventionally be viewed as the protocol (because it goes over the wire) while the client interface to the local object is the interface. Above the basic remote procedure call protocol layers, even the "protocol" can be specified as a procedural interface. More or less following convention, we use the term *protocol* to refer to that which goes over the communication facilities, and *interface* to be that visible to the client of the module in question. Thus, these two are the same in the case of a simple proxy object, and can radically differ when the stub object is a hidden component of the implementation of the client proxy object.

Separating the interface from the protocol using local programming effectively adds functionality to the local run-time libraries and often avoids the equivalent of "system calls" in the conventional operating system environment. This general direction is particularly important in a distributed system because of:

1. the high extra cost of crossing system boundaries, e.g., RPC over network.
2. the separate client processor(s). The server processing is often a critical bottleneck so off-loading it to client machines is attractive.

[4]In this vein, you can recognize DSM as data shipping and pure RPC as function shipping. Smart proxies allow you to mix the two.

3. lower memory costs reduce the need to rely on protected system services as a way to reduce the run-time library cost. (The use of shared libraries largely eliminates the multiplier effect on memory and disk space that may result from many applications using large run-time libraries. There was a time that an important role of the operating system was providing shared code.)

Replicated Objects

Smart proxies can be used to implement a replicated object, transparent to the client. For example, files may be replicated across several file servers so that the failure of one server does not make the files unavailable. Disk space may also be virtualized so that multiple nodes appear as one *virtual volume*, with the volume here being a partitioned object. Returning briefly to the caching case, the portions of an object cached on client nodes can also be regarded as, in some sense, replicating the object state, given that communication is required between the different partition or caching nodes to maintain consistency. To support the above, the proxy needs to determine the replica to read from and implement the protocol and mechanism for writing to each of the replicas in parallel, or whatever is called for by the replication protocol. In general, replicated objects require additional protocol for consistency maintenance, quorum selection, selection on access and recovery from failures. The proxy also does coordination and location mapping. This basically how cluster file systems are implemented.

Another example is the use of replicated sensors in a factory, with the client-visible object making the replicated sensors appear as one fault-tolerant sensor to the rest of the application system. Here, the smart proxy has to combine the values from multiple sensors into a single logical sensor.

Partitioned Objects

An object may be partitioned across several servers, possibly because it does not fit on one server. For example, a very large file may span multiple file servers. (The Zebra file system (SOSP93) stripes files across several servers for higher performance.) BitTorrent is a more recent example of a system using stripping, in this case, attempting to use the aggregate bandwidth across multiple endsystem hosts.

An object may also be partitioned for administrative or security reasons. For example, security keys may be fragmented across several authentication servers so that the compromise of one server does not compromise the security of the entire system.

Proxy Programming One issue arising with non-trivial proxy objects is the need for dynamic loading of the proxy code into the local address space on access to a

new type of object. Linux provides some support in this direction with dynamically linked libraries, Windows provides DLL's, and Java provides support as well.

However, another is: how does one implement smart proxies? It is not clear how the stub compiler can provide the extra functionality.

Some authors have viewed the distributed implementation of objects as motivating a new type of object or concept in the language. For example, Makpangou et al. (91) use the term *fragmented object* rather than distributed object to capture the idea of the state of the object being fragmented across servers and the client machine. They then explore language support for fragmented objects as well as the internal inter-fragment protocols. However, distributed object programming is the same as object-oriented programming except that some objects are implemented entirely or partly remotely. Thus, there does not appear to be a strong reason for new concepts.

Manual Stub Implementation

With object-oriented RPC, the *proxy implementation* is just another implementation of this interface, so it is quite feasible to implement this proxy object implementation manually, i.e. without relying on a stub compiler. Thus, an alternative implementation with OO RPC is to just provide suitable run-time support for writing marshaling and demarshaling code and receiver routines, and skip the use of a stub compiler.

This manual approach has several advantages. First, manually generated proxy code can incorporate various optimizations that require manual programming in any case. For example, a manual proxy implementation may recognize that some attribute of an object does not change over its lifetime and so can be cached in the proxy, eliminating calls over the network. In fact, the interface can provide access to some attributes as in-line functions, at least in C++. Further, a proxy for, say, an open file may cache data blocks, eliminating the network overhead when the requested file block is local. Further, a manually programmed proxy may use datagrams, multicast and asynchronous streaming, all standard techniques in networking, but not supported by the typical stub compiler. Second, manual generation avoids dependence on a stub compiler, which is a generally a large complex compiler-like piece of software that is a burden for a software team to maintain, especially given the typical "systems" programmers that use RPC are less comfortable with compiler work. For completeness, a stub compiler needs to be able to pass all the complex data structures that can occur in the language yet well-designed interfaces tend to pass very simple data units. Thus, they are often significantly complicated by facilities that are never used. All too often, projects start out using a stub compiler and then find that their communication means is restricted by the stub compiler, forcing them to either modify the stub compiler (if they are able to), abandon the use of RPC, or use a complex mixed mode of RPC and socket-level programming.

Finally, avoiding dependence on a stub compiler often makes the implementation of the RPC support more portable and more directly traceable to a wire format, as seems needed for OO RPC to make any progress in the Internet.

The disadvantage, of course, is the need to explicitly develop and maintain the proxy and receiver routines. However, in my experience, the amount of such code is small compared to the overall project, especially when one tries to use common interfaces for I/O, naming, etc. as is proper to do in any case. Moreover, any large project needs to stabilize these interfaces as early as feasible, so this code ceases to be a significant maintenance item later in a project, except for compelling optimizations and these are far easier to handle with manually generated proxies than by changing the stub compiler. Finally, with suitable run-time support, simple proxies and receiver routines are simple straight-forward code to write while with complex proxies, the vast majority of the code has to be manually written in any case, and the stub compiler frequently gets in the way.

Overall, manually generated proxies provide (almost) all the benefits of (OO) RPC, namely encapsulating the complexity of network communication, but without dependence on a stub compiler and with the option of optimization using smart proxies, reducing the performance and fault-tolerance problems with RPC. This is just that standard OOP aspect of separating interface from implementation, extending this separation compared to that provided in the single machine environment.

Residual Problems

Even with smart manually implemented proxies, the performance and fault-tolerance issues are not resolved in reality. A smart proxy that caches still can have a cache miss and then behaves the same as a simple RPC proxy. This just introduces a new source of variance and wider range of variance into the performance of a procedure, i.e. local, cached and remote.

Moreover, the fault-tolerance issues have not changed much either. If a server node fails, the proxy still confronts this issue of retry. The only change is that a smart proxy can be programmed to take advantage of the type- and function-specific semantics of the call, although that does not help in all cases. e.g. if the call is idempotent then it is fine to retry and otherwise, it may be implemented by encapsulating the call in a transaction. Again, the latter tends to increase the cost of the procedure in the common case of no failure.

Finally, in spite of the advantages of manual proxies, these still entail considerable rather dull but error-prone programming that needs to be consistently updated whenever an interface is changed. I.e. it is message-like programming, just with some framework support. Can we do better?

3.4.2 REST

*Representational State Transfer(*REST) is another approach that has been pushed for building distributed applications (Fielding's thesis). A REST distributed application is defined as one that is built according to the following principles:

1. uniform reference: there is resource identifier in a common name space for every resource (object).
2. a well-defined set of operations that are supported across all resources, e.g. the GET, POST, PUT and DELETE of HTTP.
3. (re)presentation: there is a representation of the state of the resource that is transferred so it is structured, including containing embedded references to other "resources".
4. stateless: each request from client to server is independent with no per-client state.

The pure Web model of URLs, HTTP requests and no residual client state in the server is an example of REST.

Note that, except for the last principle (and an extreme interpretation of the second) object-oriented structuring calls for similar principles, namely uniform references, well-defined interfaces and typed/structured representation. Moreover, "stateless" is largely just an implementation simplification, not really a core issue, at least IMHO. That is, REST, like NFS before it, illustrates how a "stateless" approach allows you to get some earlier success by taking an approach that is relatively easy to implement and handles an interesting subset of application requirements. However, stateless does not take you that far, as evidenced by the extensive use of *cookies* in the web. Cookies are basically per-client state, but pushed onto the client to provide back to the server on each request (so the server does not need to store this state).

REST is an attractive direction because:

- it recognizes some key difficulties with scaling, namely per-client state and overhead, and the fault-tolerance issues that come with that.
- it builds on (i.e. copies) the successful model of the Web

Considering cookies further, you can regard the issue as state coordination or consistency between the client and the server. A stateful protocol such as TCP has a connection phase that establishes state synchronization between the two ends, basically just the sequence numbers. The protocol also includes means to maintain

this consistency, namely the acknowledgment mechanism. As with the issue between datagrams versus virtual circuits, cookies can be regarded as a datagram-like mechanism of passing the state per request (similar to passing state in datagrams) versus having a setup phase that establishes this state at the other end and a tear-down mechanism that releases this state, thereby avoiding carrying the state in each message. Thus, it is basically a tradeoff between complexity and bandwidth.

However, looking at this as a state consistency issue, one significant deficiency of this stateless approach is the inability for the server to push updates to the client. This arises with the conventional Web structure, where a client needs to poll the server to detect changes at the server end, rather than being called back. Thus, to achieve rapid response to infrequent changes, this deficiency is effectively just another performance overhead.

Comparison to SOAP

SOAP is based on some of the same technology as REST, namely HTTP, and XML as an extension of HTML. However, SOAP is basically just RPC using XML as the presentation protocol and Web Services-specific protocols such as UDDI for discovery, etc.

3.4.3 Considering the Web

Looking at the web in general, you can also interpret it as a file-like access mechanism where there is structure on the file data, i.e. HTML is a presentation language. That is, rather just transferring raw bytes as DSM and distributed file systems do, you transfer structured data, structured by HTML or XML. Furthermore, rather than focusing on "shipping function", the basic web model is to ship data. The key aspect that is missing an efficient means to keep this state up to date after it has been received at the client.

Let's consider an approach to remedy this deficiency next.

3.4.4 Attributed-oriented Stateful Proxy

Here's a sketch of an approach that I developed, which has seen some serious use but is not generally known or accepted. First of all, as described in my CS249 notes, you can restrict each interface to just containing member functions that implement an *attribute*, that is a logical state variable that can be read and written. Thus, the operations in the interface are just read and write of this attribute, including operations for adding and deleting objects from collections. In this approach, each read operation is nilpotent and each write operation is idempotent.

Second, you can require that the proxy object include all the state associated with these attributes. That is, it is *stateful*. When you create a stateful proxy for a remote object, it is initialized to contain a complete copy of the interface-visible state in this remote object.

Finally, modifications to an attribute, either in the "real object" or to a writeable proxy, cause an update indicating the modification to be propagated to all the other copies of this interface, asynchronously after the write has completed. (The application can be notified by these updates using the CS249 notification approach or by polling.)

This approach provides the same basic object-oriented interface to applications, insulating them from the network/communication layer, just as with object-oriented RPC. However, there are significant differences.

First, the semantics in recovery are much clearer. In particular, when communication is lost and then restored between the the proxy and the master (either because one side failed or because the network connectivity failed), the proxy and master object just have to resynchronize the state. For example, the master can transmit its entire state to a proxy, causing a sequence of updates to the proxy to bring it into synch. There is no problem if this results in replaying some state update because these state updates are idempotent, i.e. there is no effect in redoing them.

Note that this advantage carries over to providing redundancy. A system can provide a special node that maintains all the master copies of the objects, often called an *object store* or *system database* (sysdb). Other components of the system mount object aggregates from this process either for read or read/write. All updates flow through this central repository. A backup process running this object store can be implemented as one that registers its instances as proxies for those in the primary, receiving updates just the same as another client. When the primary fails, the secondary simply converts its proxies into acting as the master copies. Moreover, when the failed primary recovers, it can simply resynchronize its state with the current object store process by reloading all the object aggregates as proxies. Thus, recovery is straight-forward. (Recovery complexity is often an neglected issue in some approaches.) Moreover, application-specific consistency is achieved by monitoring how "fresh" the attribute data is, which can be provided a timestamp, for instance. In another view, besides staleness of proxy data, a process in a distributed application just has to deal with failures in setup or mounting remote proxy data, not on every call.

Second, performance is far better. Read and write operations are always local so the problem of unpredictable performance with RPC is eliminated. In particular, a read operation just returns the state for that attribute from the local stateful proxy. An iteration reading a collection attribute occurs without any network access. Similarly, a write operation just updates this state locally. It also propagates an

update to the copies of the object, but this occurs asynchronously with respect to the caller. Also, these updates can be streamed and multicast, taking advantage of asynchronous transmission of these write-updates. In fact, this approach can model (and implement) a publish-subscribe system pattern of communication using objects. The master object acts as the publisher with updates being streamed to each proxy as a "subscriber". You can further have objects that correspond to "subjects" along the lines of conventional publish/subscribe systems.

Finally, by being a restrictive subset of programming, namely reading and writing attributes, this approach avoids coupling to specific programming languages and systems, as arises with RPC. In fact, these write-update messages are similar in structure to the type-length-value (TLV) structure of messages used in the Internet. In that vein, the proximity of these write-update messages to many Internet protocols provides some validation of the model as a general-purpose protocol model that has proven itself in the Internet.

As another view of this approach, you might regard the attribute-based interface approach as basically pushing the interface into something similar to an object-oriented database model with the stateful approach favoring so-called "data shipping" over "function shipping". I.e. there are similarities to what you might consider a distributed object-oriented database, although significant differences relative to the realizations of the latter in practice. Also, this is similar to the HTTP/HTML approach of providing structured (i.e. typed) state.

However, this approach requires dramatically restricting interfaces to just attributes, which is not the current practice. In contrast, general object-oriented programming allow what several systems, such as CORBA and UML refer to as operations. These are functions/methods in the interface that can perform arbitrary internal actions. Nevertheless, as argued in CS249, it is difficult to avoid an object interface becoming complicated, inconsistent and ill-specified over time all the while remaining incomplete for client requirements, prompting further evolution and convolution. The *attribute* model of interfaces is the best approach I have come up with for dealing with these problems. The nilpotent and idempotent restrictions are similarly a strong pill for many to swallow.

Another issue for some is creating the state in the interface. You can imagine having to pay the overhead of instantiating a large complex object proxy just to perform a simple operation where a simple request-response interaction would suffice and require far less bandwidth. However, in my experience, it is usually feasible and desirable to provide interfaces that are scaled to that required by the clients and this sort of "one off" access is infrequent and thus does not dictate performance. I.e. this case may be slower but rarely arises in my experience.

Another criticism is that this approach basically just transfers the round-trip time issue to the point of creating a proxy. That is, when an application determines it

needs access to an object, it needs to indicate to the server end that it needs the state for the proxy and then wait for this state to be transferred over the network to it because it can access this proxy object. To some degree, this is true, but by amortizing this round-trip time over the whole sequence of accesses to the object, the cost is less. Moreover, one can define object aggregates such as a manager module and all the objects that it manages as one unit that is transferred and instantiated as a proxy in one round-trip time. In this fashion, the cost of accessing a set of related objects can be amortized over a large number of objects. As a further point on this concern, these aggregated object collections can be "mounted" similar to a file system at the start of a program's execution and can be asynchronously mounted with the application notified when the (proxy) objects are instantiated locally as a result of the mount. In fact, the asynchronous notification model with stateful proxies (as I have defined them) can be carried over the mounting, so an application responds to notification when a proxy object is available in the same way as it responds to notifications when some attribute is changed.

In summary, with a sufficiently restricted interface and a stateful implementation, you can avoid the key disadvantages of OO RPC while achieving much of the benefit. I am convinced that the attributed-based stateful proxy is "the right solution" but it is so far from broad industry practice and research theology that you have to decide for yourself. Make sure you at least understand the issues and trade-offs.

3.5 OO Interface Design

Interface design is a key challenge in object-oriented distributed systems. It is the same as for non-distributed object-oriented systems, but more demanding. The interfaces corresponding to what goes over "the wire" determine how state and functionality is divided up between nodes. Their design has an enormous effect on system functionality, availability, reliability, performance, fault-tolerance and security. Related to these properties, they also affect the overall system complexity. Get these interfaces right, and it becomes a small matter of local programming to implement these interfaces correctly and efficiently to make a good distributed system. If one local implementation is weak, discard it or reimplement — a local fix. Get the interfaces wrong, and there is no fix short of a system-wide change to the interfaces.

The object interface design follows the same basic requirements of proper object-oriented programming. That is, each interface should be designed to provide:

- functional completeness.
- efficient implementation.

- PARS — performance, availability, reliability, and security.

However, because calls to the object may be remote, these considerations are that much more severe in a distributed system. For example, an interface that requires extra client calls to the object compared to a better designed interface in a non-distributed system may incur a few extra microseconds per client interaction as a result. However, in a distributed system, the extra call may add tens of milliseconds of delay and hundreds of thousands of extra instruction processing overhead to the client interaction, i.e. easily a thousand times the penalty or more. (The stateful proxy angle means this effect is restricted to the instantiation of new proxy objects.)

The design also has fault-tolerance implications, or perhaps one should say *complications*. For example, one has to deal with the scenarios like not being able to call a remote object because of a network failure. In a single-address space programming model, all objects are available at all times until deleted. Similarly, a poorly encapsulated class may cause the failure of a single program in a single address space application, but, in a distributed system, may cause a number of independent network nodes to be impeded by allowing a shared server to be corrupted.

The uniformity and evolvability requirements on interfaces in a distributed systems environment is also that much more demanding. Generally, once an interface is made available in a large-scale system, it needs to be maintained more or less indefinitely, similar to the interfaces of widely distributed software.

For instance, you should use multiple simple interfaces even for just one service rather than a single more complex interface. For example, a file service typically has a directory interface, an administrative interface and a separate open file interface. Thus, client software that purely accesses an open file that is passed to it does not need to know or depend on these other interfaces. This *need-to-know* basis for designing interfaces reduces undue dependence on interfaces. For example, the administrative interface can be extended without affecting the open file interface or any of the clients of this second interface.

3.5.1 Object References

There is a need for a uniform system-wide means of referring to objects.

I advocate a system-wide directory that maps user-assigned names to objects, the same as the model that has evolved with distributed file systems. (In fact, a file is just one example of object — we are trying to generalize from a particular example.)

To support this directory service, there needs to be a root type, say `NamedInterface`, that is defined by and used as a return type of the directory service. All other interfaces are derived from this root interface, so they can be downcast from this return type from the directory to the specific interface type, as arose with the earlier

example of casting an `NamedInterface` to an `OpenFile` interface. This root interface can also define other useful facilities. For example, in C++, the `NamedInterface` class can define a virtual destructor, ensuring the destructor has a virtual binding, and also ensuring that run-time type identification should work with instances of this class.

An alternative approach is to provide a top-level interface that supports querying what interface it supports, as done in MSFT's DCOM. I do not understand the benefit of this approach over just a dynamic cast. After all, the client needs to know the interfaces that it can use in advance, and there is never in my experience sufficiently many that one cannot just use dynamic cast to determine the one to use.

Learning again from file systems, the directory service should support hierarchical naming and the general ability to group objects into directories for convenience of listing, processing, query, etc.

3.5.2 Resource Pools and Managers

A distributed system can be regarded as different *resource pools*. For instance, each computer node is a processor, memory and network interface plus possibly other facilities such as disk storage. Distributed applications often need to pay some attention to these different resource pools because each can be depleted or fail completely individually, affecting application performance.

It is attractive to provide an interface that is instantiated to represent object implementations using different resource pools. This is commonly called the *manager* interface in conventional object-oriented programming. For example, a `OpenFileManager` interface provides the creation of open files on a particular volume.

```
class OpenFile::Manager : Directory {
public:
     Ptr<OpenFile> newOpenFile( Name );
     . . .
  };
```

Thus, a system with several different file serving nodes would have several different file manager instantiations. A naive application could be oblivious to these different file managers by simply mapping names through the directory system to file manager implementing the file, the same as for a conventional file system with multiple volumes. However, an application that is concerned about replication for fault-tolerance would check that the replicas are on different file managers and nodes in the distributed system, similarly for one that is striping a file for performance.

In the object-oriented programming world, the manager structure above is similar to the *abstract factory* and *factory* patterns described by Gamma et al. (93) and Linton's kits (Linton 92), although their focus is on single-address space design and object creation. When the object exists and uses resources in a separate address space (and normally a separate node), there is a clearer need for a manager to handle the object over its lifetime rather than simply a mechanism for creating new objects, as is the focus of kits and abstract factories. Some work has referred to this manager as *type manager* because it implements an (abstract) data type. However, the *type* of a given type may be larger than the extent of the type handled by one manager. That is, there may be multiple managers implementing the same type, e.g. different file managers managing different disks.

Each manager is also an object. This manager object is managed by a higher-level manager which is responsible for creating the managers (often on system initialization), allocating resources to these managers, and sometimes terminating and restarting them when necessary. A server node may contain one or more local top-level managers, typically one per service running on the node. In addition, there may be additional mangers that monitor the condition of each of these service managers, often serving primarily a *watchdog* functionality.

Each manager has a name and the objects it manages have names. However, these names are typically quite separate and independent portions of the name space. For example, a file system/manager may be identified by the name of the disk "/dev/dk0" which is independent of the name of the files. In fact, the full name of objects provided by a manager can be determined by *mounting* its name space in a designated part of the directory structure with only the local structure to the name space known in the manager. This requires a mapping of name prefix to manager. For example, if manager "foo" implements all the files under a directory "/web/docs", there needs to be a mapping from the prefix "/web/docs" to this manager "foo".

The notion of manager also fits the modeling and simulation perspective, because the real world employs managers to create (hire), manage and delete (fire) resources/people.

3.5.3 Recursive Structuring

Some folks view that the level of abstraction is another key design issue with interfaces. E.g., remote disk vs. remote database system. However, it is generally necessary to provide both high-level and low-level interfaces so it is the application or dependent service that, as the client, gets to choose which level of interface to use. A lower-level abstraction is simpler, faster and more primitive for wide range of uses and less cost per operation for remote node. A higher abstraction allows

better caching, contention control, scheduling and supports hardware resources concentrated in one node. One can also look for the minimum flow communication point in interface. E.g., high-level graphics display server vs. bitmap server.

The complexity of providing several levels is reduced by using the same interface to access several levels of implementation. For example, the open file interface can be used to access raw disk, a regular file or a database view.

As we know, *recursion* is a powerful structuring approach. From an interface standpoint, it allows you to reuse the same interface at different layers, rather than having separate interfaces at each layer. This reduces the number of distinct interfaces in the system, thereby reducing the design and maintenance complexity. As a minor note, this recursive approach also reduces the implementation complexity because each layer generally uses the lower layer through the same interface, reducing the number of interfaces involved with the implementation.

For example, a distributed replicated file provides the same interface as a local file and is implemented using local files. These local files are in turn implemented using hardware files, namely disks. However, each level can use the same generic I/O interface. (See Chapter 9 for more discussion.) Unix can be viewed as providing some degree of recursive structuring, because the raw disk interface provides the same open/read/write/close interface as the file object. Similarly, distributed parallel/concurrent processing facility can provide the same interface as, and be implemented in terms of, a "local" parallel/concurrent processing facility, which is in turn implemented in terms of hardware processors.

The recursive structuring has several advantages:

- conceptual simplicity and unity — the same uniform interface and the same basic service model are provided across levels of abstraction and functionality.
- implementation simplicity — less redundancy and greater compatibility of functionality between layers.
- verifiability — conducive to inductive verification because of the uniform interfaces and structuring, and because of the simplicity.

The primary complication of recursive structuring is that calls from the lower layers to the upper layers can result in a recursive loop unless special cases are provided to terminate the recursion. For example, consider a disk module that is logging disk errors to a file.

This problem is exactly the same as arises with, for example, the classic factorial function

```
factorial(n) = n*factorial(n-1) except factorial(1) = 1
```

If the `factorial(1)` special case is not provided, the execution of factorial produces an infinite recursive loop. In our disk example above, we can recognize that logging disk errors to a file is fine except for disk errors on the disk containing the log file. A special case is required to handle this situation.

Conventionally, computer system designers and programmers in general are taught to avoid special cases. However, the special cases in recursive structuring are justified based on the overall benefits of recursion described above.

3.5.4 Example Interface: UIO

To illustrate some of the design decisions that arise with interface design, let's consider the UIO interface developed in the V system. This interface is a bit old at this time but I/O is still central to systems design, it is a familiar "application" and the design decisions and lessons are still relevant.

The UIO interface was developed in V to provide a uniform or common interface to all I/O-like services. Using UIO, operations that are sufficiently similar across all I/O services so programs can be *relatively independent* of final I/O binding. Without this, each program is specialized for different I/O services, which leads to more programming, complexity and less flexible interconnection. This is of particular concern in distributed systems where there can be more variety and extensibility.

The UNIX I/O is a good start but deficient, e.g. problems with distributing the shared file pointer (which is per-client shared state).

As background, in Thoth, the I/O system design was poor; there was no clear protection boundary and ineffective use of the messaging. A careful design of I/O was clearly needed. Redesign goal was: Handle wide variety of I/O devices in uniform fashion.

First attempt: copy WFS design (See Symp. on Op. Sys. Principles, 1979). (Note: WFS might be regarded as the inspiration to NFS). e.g., ReadPage(file, page, buffer, bytes) Here, there was no open, close, no open file connection, so no connection overhead, a so-called 'stateless' interface. Problem: How to deal with objects created by 'opening' connections, e.g., TCP connection, pipes.

Second attempt: transient object created, called UIO, which emphasizes 'creating' rather than 'connect to'. In the object-oriented view, we want to return a pointer to an interface that provides operations to read and write the file. That is, the important recognition is that 'opening an object' is really creating another object. The object provides stateful interface to file access.

This I/O interface can be modeled by a C++ or Java interface (base) class. However, a programming language does not specify the generic semantics of such an interface.

Challenge: define an interface for I/O to lots of devices, capturing what is in common and handling the differences.

UIO Overview

UIO model is structured as a basic model (compulsory functionality) plus hierarchy of *optional* functionality as well as standard *escape* mechanism.

The basic UIO object defines interface attributes for the state, consisting of control and sequence of data blocks. The control includes identifier, owner, authorization, permissions, and data block description in terms of blocksize, nextblock, lastblock, lastbyte plus attributes indicating variations in UIO functionality.

Each I/O service abstracted as creating, reading, writing, releasing and (control) querying and modifying UIO objects.

We rely on run-time libraries, such as an implementation of the C standard library, to map from the language standard I/O primitives to UIO objects. In particular, a run-time library can easily implement byte I/O to provide a byte stream abstraction on top of UIO objects.

Let's discuss some design issues!

Data Access Model

The *data access model* is the specification of the unit of data to read/write, how to address, and constraints such as how many, what order, etc. E.g., single byte sequential vs. multi-block random access.

UIO: *block* as unit, addressed by 32-bit block number, and: sequential, random access, variable length, etc. depending on attributes.

Why block (vs. bytes)?

1. Models logical records, e.g., record files, networks, terminals.
2. Efficient unit of transfer.
3. Simplifies caching.

I regard the use of the byte-stream abstraction at the system interface, such as in Unix/Linux files and TCP streams, as a fundamental design mistake because it throws away information, namely the unit boundaries, which some applications care about, for performance or semantics. I.e. you either receive a packet or you dont, giving different semantics to a packet-aligned/contained message that one that is not. It is difficult to get this information back once the lower-layer has provided the

byte stream interface. However, it is easy to "paper over" a block stream interface with a layer that provides a byte stream abstraction.

The block stream also allows larger addressing range than byte addressing for the number of bits in the block address, assuming blocks are larger than one byte.

Why fixed-size (e.g. 32-bit) block address?

1. adequate for conventional (unstructured) file access.
2. sequence number for sequential I/O (idempotency).
3. enumerating records in a DB *view*, handling key specification as part of UIO creation (view creation), rather than on every read.

Another issue is how to handle bi-directional streams such as a TCP connection. The approach is to support bi-directional streams such as a TCP connection by providing a second UIO object for the other direction and providing an operation in the extended interface to return this other object.

With sparse files, the nextblock and lastblock as lower and upper bounds on the valid blocks. A "invalid block" error is returned on an attempt to read a missing block. The structure on the file should indicate which blocks are valid, so invalid reads should not be a problem.

Note: block number in each Read/Write $\Rightarrow$ idempotent. Idempotency allows a file server to respond to a read request without buffering the data for retransmission because a retransmitted request can be handled as a new request and just regenerate the data.

I/O Access State

The UIO I/O access or control state is associated with access to data, not the (persistent) state or data of file or device itself. E.g., version of file for atomic update, pipe, window, net. connection state.

UIO object provides a stateful interface for I/O access in the form of an object that is accessible by remotely. Passing access to an open file between processes on different hosts requires using a separate name for designating the UIO object and having the receiving machine create a new proxy object for this UIO object.

Claim: *stateless* interfaces are too limited. For example, consistent caching implies state to remember who has copies of the data, to be notified when it needs to be invalidated.

A stateful approach also allows the called object to determine the (security) principal of the caller, so the I/O mechanism can check permissions on object creation. Also,

the OO RPC system can also use connection state (and heartbeats) to detect when the client has failed, allowing it to reclaim this UIO object. If the OO RPC runs over a connection-oriented protocol, a common situation, the principal can be associated with the connection on connection setup.

The standard criticism of using a stateful interface is that the client application fails when the server fails and reboots. However, one can make this state recoverable by having the clients resupply the state to the server after a server failure and recovery. In the case of read-only files, the client can just reopen the file and seek to the right position to recreate this state. In the case of modifying a file, the client can do the same, possibly rolling back to the last committed write, but the same problem arises with stateless interfaces. Also, the client can retain file blocks until the server has committed them to disk, so they are not loss in a crash.

Once the client is able to recover from server failure, the server can handle running out of memory consumed by client state by simply "forgetting" access state to files and have the client recreate it, relying on the above recovery mechanism. As a heuristic, the server can select least-recently access open files. Moreover, given the low price of memory, a file server can be configured with a sufficient amount of memory that it can accommodate open files far in excess of its I/O capacity so will only encounter this problem when many are idle for significant periods of time.

Base and Extended I/O Functionality

There is a basic functionality that is compulsory, providing the base operations/semantics and standard return exceptions plus additional properties as indicated by special attribute flags. This is outlined in the interface below.

```
class UIO : NamedInterface {
public:
    virtual Block block( uint blockNo, size_t bytes ) const = 0;
    virtual Block newBlock() = 0;
    virtual void block( u32 BlockNo, Block ) = 0;
    virtual size_t blockSize() const = 0;
    virtual u32 nextBlock() const = 0;
    virtual u32 lastBlock() const = 0;
    virtual u32 lastByte() const = 0;
    enum UioFlags {Resettable = 1, RandomAccess=2, Storage=4, FixedLength=8,
          MultiBlock=0x10, VariableBlock=0x20, InsertDelete=0x40,
          Locked=0x80, IntentLocked=0x100, Snapshot=0x200, Recoverable=0x400,
          Replicated=0x800, Interactive0x=1000};
    virtual u32 attributes() const = 0;
  };
```

The UioFlags are a reflection of I/O stream library requirements. I have not found a grand unifying theory for these.

The base functionality of a UIO object is a sequential read or write I/O object, e.g. one side of a TCP stream with records. The UIO flags indicate the additional semantics, as outlined below.

- Resettable - e.g. rewind the tape. Access to blocks is sequential, except for being able to specify block 0 and return to the beginning of the sequence of data blocks.
- RandomAccess - access to blocks can be in random order, as opposed to just sequential.
- Storage - one can reread same data or read what was last written to a block.
- FixedLength - The amount of data in the file is fixed, so a read returning less than what was asked for is an error or means end of file.
- MultiBlock - the UIO supports multi-block reads and writes. Otherwise, the interface only supports reading or writing up to a blockSize at a time.
- VariableBlock - blocks other than the last block in the sequence can be less than the block size.
- InsertDelete - blocks can be added and removed in the middle of the data block sequence, leading to a sparse file.
- Locked - the file is locked.
- IntentLocked - the file is locked with an intention lock.
- Snapshot - the file data as represented by its data block sequence will not change during the time that the UIO object exists.
- Recoverable - the UIO object can be reset back to its earlier state.
- Replicated - the UIO is implemented on several servers.
- Interface - it represents a user interaction device such as a key board, so controlling character echo, etc. makes sense.

Providing these attribute flags allows the client to simply and efficiently test for these capabilities rather than having to write code “experiment” to see if they are support or else assume that an object has this capabilities and then handle the error when it fails on an operation.

Specification of I/O Service

What's a uniform way to specify one of many services, service options, etc.

Solution: use character string name to specify, with details buried in name *definition*, rather than special options.

Alternative: name encodes the definition (description) or some parameters. For example, the name could be "/net/tcp/36.8.0.8".

This approach fits the general model of creating objects by name, including the approach to files and the name handling in current programs. I.e. the program passing around a name, not a lot of other parameters. Also, from a performance standpoint, name mappings get used repeatedly so having to first create the name definition and then do the object creation in this design is infrequent. Finally, as we shall see in Chapter 5, the name lookup can be decentralized to individual object modules including specific I/O modules, allowing device or type-specialized mappings of name to description. (This type specific information has to be passed into the object module in some way, in any case.)

Locking

There has been relatively little experience with locking as part of the UIO interface, but this seems like a good place to support locking because:

1. Reduced communication costs — I/O server is best place for synchronization and lock/unlock as side-effects of I/O operations.
2. Semantics: we can then define interactions between locked and unlocked access. (Part of OOP thinking is defining the semantics of an interface w/o regards to implementation.)
3. coupled to atomic transactions and recoverability.

Two forms of UIO locks: file and block.

File/UIO lock on Create.

Intention lock for block-level locking (specified in mode).

Locks on two different UIOs versus the same UIO? That is, are we locking the file or the UIO?

Piggybacking lock acquisition with intention-locked UIOs.

Automatic lock release on UIO destruction.

Also, explicit LockUio and UnlockUio operations.

Pros:

1. minimizes comm. overhead for 2-phase locking, e.g. in conjunction with file transactions.
2. easily implementable in server.
3. adequate client facility for atomic transactions.

Atomic Transaction Support: support for creating a UIO object that is a clone of the UIO state for nesting transactions.

Otherwise, transaction management is a separate protocol, such as the object-oriented transaction protocol (OOTP) of Chapter 7.

Other Issues

Replication: the implementation is handled in the proxy object that implements (say) the read-any, write all available protocol. Replicated file update is best handled as part of an atomic transaction because one replica can fail during update.

Mapped I/O: UIO object is the object bound into the address space using memory-mapping supported by the VM system. For ease of implementation, this mapping might be restricted to UIO objects that have a block size that is compatible with VM page size. In this vein, an application can have three modes of access, namely byte, block and memory-mapped with the first supported through the I/O library, the last by the VM system and only the block access directly supported by the UIO interface.

Specialized operations:

Options have been strictly avoided in standard operations so common client software does not need to know about them. Originally, the UIO interface used a standard UioControl operation as a standard escape for special-purpose operations, the same as the Unix ioctrl. However, the preferred OOP approach is to allow each service to support an extended interface with additional specific operations. E.g. an extended interface for a terminal might look like:

```
class TerminalUio : public Uio {
public:
   enum EchoMode {EchoOff = 0, EchoOffOnce = 1, EchoOn = 2};
   void echo( EchoMode ) = 0;
   EchoMode echo() const = 0;
};
```

The sophisticated client then dynamically casts the object to the specialized type to test when it does have this more specialized interface and to gain access to these specialized operations, if so.

```
Ptr<Uio> uio = . . .
if( TerminalUio * tu = dynamic_cast<TerminalUio *>( uio.ptr() ) ) {
    tu->echo( TerminalUio::EchoOff );
  }
```

Note: you can regard the use of the flags to reduce the number of derived classes, particularly when different capabilities do not lead to different interfaces. I.e. you really only want to use a separate derived class when the capability requires an extension to the interface.

In some cases, an option can be set by one of these separate operations to affect a subsequent standard operation. For example, you can turn off character echoing for the next operation only (by writing `EchoOffOnce`) and then issue the read to a terminal device. For efficiency, the turning off the echo can reset automatically after the read completes, what I call *auto-reset*.

V Experience with UIO Interface

The UIO interface was used by a diverse set of servers in the V distributed system. Client software was generally written in terms of C stdio, provided by a simple, efficient client run-time library that mapped to the UIO interface. Performance is good, dominated by byte-level overhead if byte-stream.

More experience, servers, exotic uses would be useful to have. However, it did seem very promising as an interface fitting into the OOP thinking and providing a uniform access to a much greater diversity of devices than conventional file access protocols, such as NFS, etc.

3.6 Concluding Remarks

The top-level challenge is providing a framework or infrastructure for implementing distributed applications which can reduce the overhead that applications experience in dealing with the gore of communications while allowing efficient fault-tolerant operation. Conventional approaches such as DSM and distributed file systems provide unstructured (raw byte) data and, in the case of file systems, usually lack incremental push updates, i.e any real support for efficient consistency. Communication-based approaches (i.e. messages) tend to leave a lot of work for applications, basically the mapping of state to and from the communication layers, namely marshaling and demarshaling. In all these cases, there is excessive distance between the programming language handling of state and that provided by these mechanisms.

Remote procedure call (RPC) cures this “distance” problem, providing automatic conversion between a language construct, namely procedures and the communica-

tion layer. However, the procedure model suffers from poor failure semantics, performance problems and excessive coupling to language-specific issues. In essence, it is the wrong language structure to focus on because it is focused on procedural, not state, and state is the central issue for distributed systems.

The best approach is far from settled. My proposal is a stateful proxy approach in which a file-like but structured and typed object aggregate can be "mounted" and instantiated as proxies in a stateful way that supports subsequent push state updates from the server.

Interface design in distributed systems is even more critical than with smaller-scale object-oriented programming. We touched on using manager objects and generic or uniform interfaces to reduce entropy and allow for interoperability.

The object-oriented approach provides a mechanism for implementing the spectrum of state sharing techniques identified in the previous chapter, with object specialization to state allowing each type of object to maintain sufficient consistency for the applications while maximizing performance and fault-tolerance. A key issue with distributed object programming is consistency between different distributed portions of the distributed object implementation. Ideally, the whole discussion of shared state would be phrased in terms of distributed object programming. We also discussed the recursive structuring approach that provides a particular model of how the layers of protocols interact.

Most existing systems are structured outside of the object-oriented paradigm. However, these designs can be mapped to the approach described in this chapter. I believe you get a better understanding of these systems by viewing them in the framework laid out here.

Many of the following chapters of the book consider the design of interfaces and protocols for specific types of objects in a distributed system. In these discussions, a key issue is the consistency issue that arises from the shared state view of distributed systems. These chapters also use existing systems as examples, pointing out how they are consistent with, and how they depart from these principles, and the consequences.

Readings

G.S. Raj, A Detailed Comparison of CORBA, DCOM and Java/RMI, http://my.execpc.com/ gopalan/misc/compare.html

References

A. Birrell, G. Nelson, S. Owicki, E. Wobber, Network Objects, SOSP93,p. 217-230

A. Barak, Mosix, www.mosix.org, 2005.

Wikipedia, Representational State Transfer, http://en.wikipedia.org/wiki/REST

D. Cheriton, UIO: A Uniform I/O System Interface for Distributed Systems, ACM TOCS, May 1987.

S. Vinoski, CORBA: Integrating Diverse Applications within Distributed Heterogeneous Environments.

M. Nelson, M. Linton and S. Owicki, A Highly Available, Scalable ITV System, ACM Symposium on Operating Systems Principles, Dec, 1995.

E. Berglund and D. Cheriton, Amaze: A Multiplayer Game Program, IEEE Software, May 1985.

M. Makpangou, Y Gourhant, J.P. LeNarzul, M. Shapiro, Fragmented Objects for Distributed Abstractions. IEEE Software, to appear.

M.P. Zeleznik, A Portable, Network-Transparent Communication System for Message-Based Applications, 6th ICDCS, May 1986, Cambridge, MA.

Hidden Difference: Doing Business with the Japanese, E.T. Hall and M.R. Hall, Anchor Press/Doubleday Press, 1987.

A. Jones, The Object Model, Operating Systems: An Advanced Course, Springer-Verlag, 1979.

J.H. Saltzer, D.P. Reed and D.D. Clark, End-to-End arguments in systems. ACM Trans. on Computer Systems, Nov. 1984, Vol. 2, No. 4

M. Linton and D. Pan, Interface Translation and Implementation Filtering, C++ Conference, April'94, Cambridge, MA

M. Linton, Encapsulating a C++ Library, Usenix C++ Technical Conf, Portland, August 1992

E. Gamma, R. Heim, R. Johnson, J. Vlissades, Design Patterns: Abstraction and Reuse of Object-oriented Design, Proc. ECOOP'93 Conf. Springer-Verlag Lecture Notes in Computer Science, 1993.

K. Harty and D. Cheriton, Application-Controlled Physical Memory using External Page Cache Management, ASPLOS'92, ACM, Boston, October 1992.

S. Liang and D. Cheriton, TCP-SMO: Extending TCP to Support Medium-scale Multicast Applications, IEEE InfoComm, 2002.

S. Liang, Unifying the Transport Layer of a Packet-Switched Internetwork, Stanford University, Ph.D. Thesis 2003.

T. Anderson et al. Tools for the development of application-specific virtual memory management. OOSPLA, 1993.

OMG, The Common Object Request Broker: Architecture and Specification, OMG Document Number 91.12.1, Revision 1.1, December 1991.

M. Zelesko and D. Cheriton, Specializing Object-oriented RPC for High-Performance Applications, ICDCS, May 1996.